

Aufbau einer Testinfrastruktur

Einleitung

Im Rahmen des geforderten Projekts möchte ich ein technisches Projekt ausarbeiten, das an meinem Arbeitsplatz umgesetzt werden kann. In einer ersten Phase besteht dieses Projekt aus dem Aufbau einer Testinfrastruktur mit Geräten wie Servern und Switches.

Bedarf

Dieser Bedarf ergibt sich daraus, dass wir innerhalb unseres Teams nur begrenzte Kenntnisse über unser Netzwerk haben, da es für uns schlicht nicht direkt zugänglich ist. Alles, was unsere Server- und Netzwerkgeräte betrifft, läuft über das BIT. Gleichzeitig verfügen wir im Team über genügend IT-Kompetenzen, um unsere Systeme weiterzuentwickeln. Mit dieser Testinfrastruktur könnten wir ein auf unsere Bedürfnisse zugeschnittenes Monitoring-System aufbauen. Das bedeutet, dass wir gezielt diejenigen Dienste, Ports und Protokolle überwachen könnten, die für uns relevant sind.

Dabei ist auch zu berücksichtigen, dass das BIT für die Umsetzung definitiver Lösungen zuständig ist, jedoch nicht jederzeit für die Bearbeitung unserer Probleme zur Verfügung steht. Dadurch können gewisse Aufgaben sehr viel Zeit in Anspruch nehmen, teilweise so viel, dass Projekte schliesslich aufgegeben werden. Mit dieser Initiative könnten wir unsere Systeme in einer gesicherten Infrastruktur untersuchen, ohne das produktive Arbeitsnetzwerk zu gefährden, und dadurch Lösungen entwickeln, die wir anschliessend dem BIT vorschlagen können.

Projektumfang

Für diese Arbeit konzentriere ich mich darauf, die Inbetriebnahmephase testbar, realisierbar und reproduzierbar zu gestalten. Ziel ist es, eine möglichst realitätsnahe Netzwerkinfrastruktur aufzubauen, um Prototypen für Monitoring und Automatisierung zu entwickeln, die später möglicherweise in der produktiven Umgebung eingesetzt werden können. Um keine vertraulichen oder technischen Informationen über unsere produktive Infrastruktur offenzulegen, verwende ich in dieser Dokumentation ausschliesslich einfache und verbreitete Protokolle zur Veranschaulichung.

Ziele

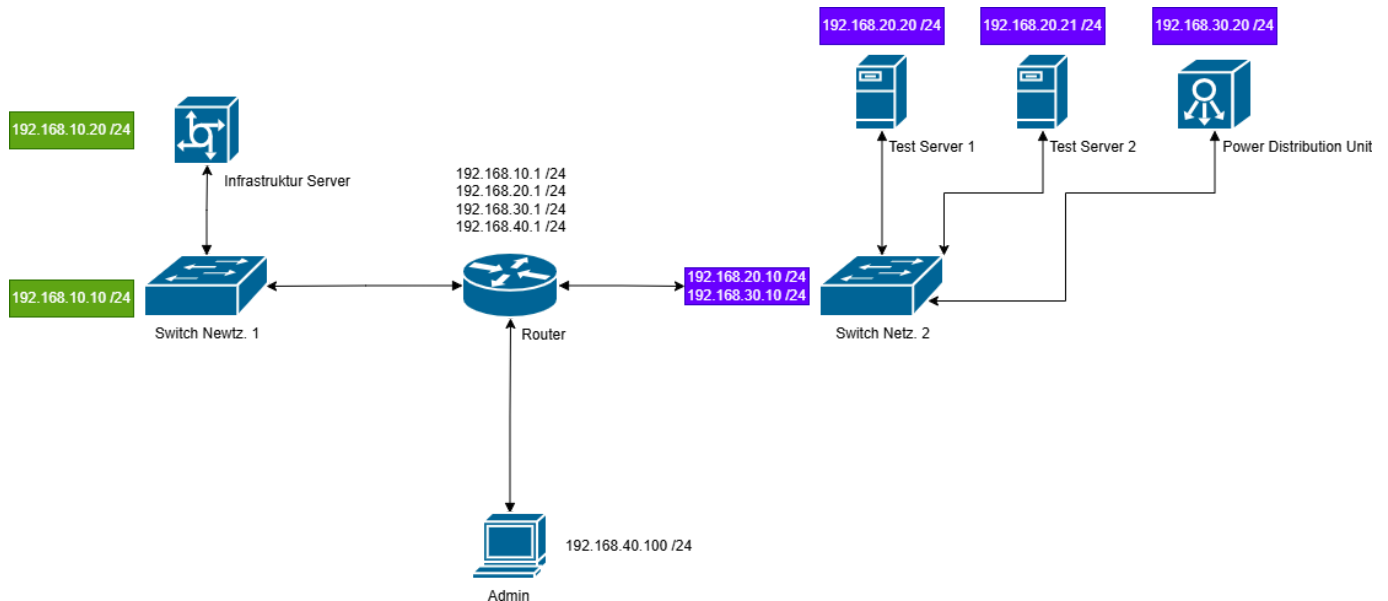
Um das Projekt übersichtlich zu halten, definiere ich drei Hauptziele, anhand derer die Idee bewertet werden kann:

1. Definition der Netzwerkarchitektur
2. Definition der zu überwachenden Elemente
3. Prototypen für die Automatisierung der Infrastruktur

Definition der Netzwerkinfrastruktur

Im folgenden Schema stelle ich die minimale Hardware-Infrastruktur dar, die für Monitoring und Automatisierung benötigt wird. Da wir keinen Zugriff auf die Firewall-Regeln des produktiven Netzwerks

haben, werden in diesem Prototyp keine produktiven Firewalls berücksichtigt. Für die aktuellen Monitoring- und Automatisierungsziele sind diese zunächst nicht relevant.



Wir unterscheiden vier verschiedene Netzwerke auf Basis des privaten Adressbereichs **192.168.x.x/24**.

Netzwerk 1

Das Netzwerk **192.168.10.0/24** enthält die Monitoring-Lösungen. Es kann über den zentralen Router mit dem Testbereich kommunizieren sowie mit dem bzw. den Admin-PCs im Netzwerk **192.168.40.0/24**.

Netzwerk 2

Wie im Netzwerk 1 gibt es hier einen Switch sowie zwei Testserver. Die Testserver können mit ihrem Gateway sowie mit den Admin-PCs kommunizieren. Dies ist für Automatisierungen sowie für das Senden von Logs an den Infrastrukturserver notwendig.

Zusätzlich gibt es eine PDU (Power Distribution Unit), die sich nicht im gleichen Netzwerk wie die beiden Testserver befindet. Dies entspricht der Struktur unserer produktiven Umgebung, in der diese Gerätetypen ebenfalls voneinander getrennt sind. Die PDU wurde in das Projekt aufgenommen, da auch hier interessante Monitoring- und Automatisierungsmöglichkeiten bestehen.

Dies erfordert eine entsprechende Port- bzw. VLAN-Konfiguration auf dem Switch des Testbereichs. Die PDU darf nur vom Infrastrukturserver beziehungsweise vom Administrationsnetz erreicht werden. Zugriffe aus dem Testserver-Netz werden blockiert.

Admin-PC

Der Admin-PC hat Zugriff auf alle Netzwerke und Geräte, damit notwendige Konfigurations- und Wartungsarbeiten durchgeführt werden können.

Router

Der Router verbindet die vier benötigten Netzwerke. Die Kommunikation zwischen dem Infrastruktur- und dem Testnetz soll auf das absolut notwendige Minimum beschränkt werden. Gleiches gilt für die Kommunikation mit der PDU.

Dafür möchte ich eine Linux-Maschine mit drei LAN-Ports einsetzen, an die die Switches der Netzwerke 1 und 2 sowie der Admin-PC angeschlossen werden. Dadurch sind die Hauptbereiche physisch voneinander getrennt. Linux übernimmt das Routing zwischen den Netzwerken. Die Zugriffsbeschränkungen werden mit `nftables` umgesetzt.

Definition der zu überwachenden Elemente

In der ersten Phase möchte ich die Testserver sowie die PDU im Testbereich überwachen. Insbesondere sollen die definierten Kommunikationswege innerhalb des Testbereichs und zwischen Test- und Infrastrukturnetz geprüft werden. Ziel ist es, beispielsweise TCP-basierte Dienste und Verbindungen zu überprüfen und auftretende Fehler zu erkennen. Diese Fehler sollen an den Infrastrukturserver im Netzwerk 1 übertragen und dort ausgewertet werden.

Zusätzlich möchte ich den Switch des Testbereichs überwachen. Fällt dieser aus, können die Testserver nicht mehr mit dem Infrastrukturserver kommunizieren. Deshalb erfolgt diese Überwachung direkt vom Infrastrukturserver aus.

Nach demselben Prinzip könnte auch der Switch im Infrastrukturnetz überwacht werden. Dies ist jedoch nicht Teil des ersten Prototyps. Ebenso wird der Router im Prototyp nicht durch ein externes Gerät überwacht. Da der Admin-PC direkt mit dem Router verbunden ist, können Ausfälle oder Fehler in dieser Phase manuell analysiert werden.

Konkrete Ziele

Bereich	Ziel	Kommunikation / Technik	Erwartetes Verhalten
Internes Monitoring	Kommunikation zwischen den Testgeräten überwachen	ICMP / TCP	Erlaubte Verbindungen müssen funktionieren
Testserver ↔ Testserver	Verbindung zwischen den Testservern prüfen	ICMP / definierte TCP-Ports	Verbindung erfolgreich
Testserver → Switch	Erreichbarkeit des Switches prüfen	ICMP, optional SNMP	Switch muss erreichbar sein
Testserver → PDU	Netzwerksegmentierung überprüfen	ICMP / TCP	Verbindung muss blockiert sein
Fehlererfassung	Kommunikationsfehler lokal protokollieren	Python-Skript / Logs	Nur Fehler und Abweichungen werden gespeichert
Log-Übertragung	Fehler einmal täglich an den Infrastrukturserver senden	TCP / definierter Log-Port	Nur vorhandene Fehler werden übertragen
Externes Monitoring	Testnetzwerk vom Infrastrukturserver überwachen	ICMP / TCP / SNMP	Testserver und Switch müssen erreichbar sein
SSH	Zugriff vom Infrastrukturserver auf die Testserver	TCP / 22	SSH nur vom Infrastrukturserver erlaubt

Bereich	Ziel	Kommunikation / Technik	Erwartetes Verhalten
Automatisierung	Testserver zentral konfigurieren	Ansible über SSH	Playbooks werden vom Infrastrukturserver ausgeführt
Standardregel	Nicht benötigte Kommunikation blockieren	nftables	Alles nicht explizit Erlaubte wird blockiert

Die folgenden Tests dienen dazu, diese Ziele zu überprüfen:

Ziel	Wie wird es getestet?	Mittel / Werkzeuge	Erwartetes Ergebnis
Testserver ↔ Testserver	Von Testserver 1 die Verbindung zu Testserver 2 prüfen	ping, nc, Python-socket	Der Zielservers ist über die erlaubten Verbindungen erreichbar
Testserver → Switch	Regelmässig prüfen, ob der Switch erreichbar ist	ping, optional snmpget	Der Switch wird als erreichbar erkannt
Testserver → PDU gesperrt	Von einem Testserver absichtlich versuchen, die PDU zu erreichen	ping, nc, curl je nach Dienst	Die Verbindung wird blockiert
Interner Fehler erkannt	Testserver 2 ausschalten oder Netzkabel trennen	Python-Monitoring-Skript	Testserver 1 erkennt den Fehler und erstellt einen Logeintrag
Switch-Ausfall erkannt	Switch im Testnetz ausschalten oder trennen	ping, Python-Skript	Geräte hinter dem Switch werden als nicht erreichbar erkannt und der Ausfall wird protokolliert
Nur Fehler protokollieren	Zuerst Normalbetrieb testen und danach absichtlich einen Fehler erzeugen	Python, Logdateien	Im Normalbetrieb wird kein Fehler gemeldet, nur Abweichungen werden gespeichert
Tägliche Log-Übertragung	Einen Fehler erzeugen und anschliessend die tägliche Übertragung ausführen	systemd timer, cron, Python	Die gesammelten Fehler werden einmal täglich an den Infrastrukturserver übertragen
Log-Empfang auf dem Infrastrukturserver	Nach einem bekannten Fehler die empfangenen Logs prüfen	Python, journalctl, Logdateien	Zeitpunkt, Gerät und Fehlertyp sind korrekt gespeichert
Infrastrukturserver → Testserver Monitoring	Einen Testserver ausschalten	ping, TCP-Prüfung, Python	Der Infrastrukturserver erkennt den Ausfall

Ziel	Wie wird es getestet?	Mittel / Werkzeuge	Erwartetes Ergebnis
Infrastrukturserver → Switch Monitoring	Den Switch ausschalten	ICMP, SNMP	Der Infrastrukturserver erkennt, dass der Switch nicht erreichbar ist
TCP-Dienst-Monitoring	Einen überwachten Dienst auf einem Testserver stoppen	<code>nc</code> , Python- <code>socket</code> , <code>systemctl stop</code>	Der Infrastrukturserver erkennt, dass der Dienst bzw. Port nicht mehr erreichbar ist
SSH Infra → Testserver	Vom Infrastrukturserver eine SSH-Verbindung zum Testserver aufbauen	<code>ssh</code>	Die Verbindung ist erfolgreich
Nicht erlaubtes SSH	Von einem nicht autorisierten Gerät SSH zum Testserver versuchen	<code>ssh</code>	Die Verbindung wird durch <code>nftables</code> blockiert
Ansible-Automatisierung	Vom Infrastrukturserver ein einfaches Playbook ausführen	<code>ansible-playbook</code>	Die gewünschte Änderung wird automatisch auf dem Testserver durchgeführt
Ansible-Ergebnis prüfen	Das Resultat direkt auf dem Testserver kontrollieren	<code>cat</code> , <code>ls</code> , <code>systemctl</code> usw.	Der Zustand des Testservers entspricht der Definition im Playbook
Default-Deny mit <code>nftables</code>	Mehrere nicht vorgesehene Verbindungen testen	<code>ping</code> , <code>nc</code> , <code>ssh</code> , <code>curl</code>	Alle nicht explizit erlaubten Verbindungen werden blockiert

Das Projekt gilt als erfolgreich, wenn folgende Bedingungen erfüllt sind:

- Nur die explizit erlaubten Kommunikationsverbindungen zwischen den verschiedenen Netzwerken funktionieren. Nicht erlaubte Verbindungen werden durch `nftables` blockiert.
- Das lokale Monitoring auf den Testservern funktioniert und erkennt Fehler bei den definierten Kommunikationswegen.
- Die Testserver übertragen einmal täglich die erkannten Fehler in Form von Logs an den Infrastrukturserver.
- Kleine Ansible-Automatisierungen können vom Infrastrukturserver aus erfolgreich auf den Testservern ausgeführt werden.

Kommunikation zwischen den Geräten

Zur Veranschaulichung der Kommunikation zwischen den Geräten und Netzwerken folgt ein mögliches Beispiel einer `nftables`-Konfiguration:

```
table inet filter {
```

```
chain forward {
    type filter hook forward priority 0;
    policy drop;

    # Nur Antworten auf bereits erlaubte
    # Verbindungen zulassen
    ct state established,related accept

    # =====
    # INFRA -> TESTSERVERS
    # =====

    # SSH + Ansible
    ip saddr 192.168.10.20 \
        ip daddr 192.168.20.0/24 \
        tcp dport 22 accept

    # Ping für das Monitoring
    ip saddr 192.168.10.20 \
        ip daddr 192.168.20.0/24 \
        icmp type echo-request accept

    # =====
    # TESTSERVERS -> INFRA
    # =====

    # Senden von Logs / Fehlermeldungen
    ip saddr 192.168.20.0/24 \
        ip daddr 192.168.10.20 \
        tcp dport 5000 accept

    # =====
    # INFRA -> PDU
    # =====

    # Verfügbarkeitsprüfung
    ip saddr 192.168.10.20 \
        ip daddr 192.168.30.20 \
        icmp type echo-request accept

    # SNMP-Monitoring
    ip saddr 192.168.10.20 \
        ip daddr 192.168.30.20 \
        udp dport 161 accept

    # =====
    # ADMIN
    # =====

    # SSH vom Admin-PC
```

```

        ip saddr 192.168.40.100 \
        tcp dport 22 accept
    }
}

```

Zusammenfassend werden nur die benötigten Datenflüsse erlaubt und die verwendeten Ports und Protokolle explizit definiert. Die folgende Tabelle zeigt die aktuell vorgesehenen Kommunikationsmöglichkeiten:

Funktion	Protokoll / Port	Quelle	Ziel	Zweck
SSH / Ansible	TCP / 22	Infrastrukturserver	Testserver	Administration und Automatisierung
ICMP	ICMP	Infrastrukturserver	Testserver / Switch / PDU	Verfügbarkeitsprüfung
SNMP	UDP / 161	Infrastrukturserver	Switch / PDU	Monitoring von Netzwerkgeräten
Logs / Fehlermeldungen	TCP / 5000	Testserver	Infrastrukturserver	Übertragung von Logs und Fehlern
Administration	TCP / 22	Admin-PC	Netzwerkgeräte	Manuelle Konfiguration und Wartung
Standardregel	Alle	Alle	Alle	Standardmässig blockiert

In einer ersten Implementierungsphase wird SSH für das Monitoring und die Automatisierung der Testserver eingesetzt. Weitere Protokolle wie ICMP, SNMP und definierte TCP-Ports werden schrittweise für zusätzliche Monitoring-Funktionen ergänzt.

Prototypen für die Automatisierung der Infrastruktur

Für die erste Phase, die sich auf Netzwerküberwachung und Log-Übertragung konzentriert, folgt ein konkretes Beispiel für die automatische Übertragung von Logs an den Infrastrukturserver.

Hier ist das Python-Skript, das auf dem Testserver ausgeführt wird:

```

import socket
from pathlib import Path

INFRA_SERVER = "192.168.10.20"
INFRA_PORT = 5000
LOG_FILE = "/var/log/test-monitor/errors.log"

log_path = Path(LOG_FILE)

if log_path.exists() and log_path.stat().st_size > 0:
    data = log_path.read_text()

```

```
with socket.create_connection((INFRA_SERVER, INFRA_PORT), timeout=10) as sock:
    sock.sendall(data.encode("utf-8"))

print("Logs erfolgreich gesendet.")
else:
    print("Keine Fehler zum Senden vorhanden.")
```

Das Skript wird mit Ansible verteilt und so konfiguriert, dass es einmal täglich ausgeführt wird:

```
- name: Log-Sender verteilen
  hosts: testservers
  become: true

  tasks:

    - name: Python-Skript für den Logversand kopieren
      ansible.builtin.copy:
        src: send_logs.py
        dest: /usr/local/bin/send_logs.py
        mode: '0755'

    - name: Monitoring-Logverzeichnis erstellen
      ansible.builtin.file:
        path: /var/log/test-monitor
        state: directory
        mode: '0755'

    - name: Fehler-Logdatei erstellen
      ansible.builtin.file:
        path: /var/log/test-monitor/errors.log
        state: touch
        mode: '0644'

    - name: Logs einmal täglich senden
      ansible.builtin.cron:
        name: "Monitoring-Fehler an Infrastrukturserver senden"
        minute: "0"
        hour: "23"
        job: "/usr/bin/python3 /usr/local/bin/send_logs.py"
```

Auf dem Infrastrukturserver wird ein kleiner Dienst benötigt, der auf TCP-Port 5000 lauscht:

```
import socket

HOST = "0.0.0.0"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind((HOST, PORT))
```



```
server.listen()

print(f"Listening on TCP/{PORT}")

while True:
    conn, addr = server.accept()

    with conn:
        data = conn.recv(65535)

        if data:
            with open("/var/log/testservers.log", "a") as log:
                log.write(data.decode("utf-8"))
                log.write("\n")
```

Damit kann ein Skript erstellt und automatisiert auf die Testsysteme verteilt werden. Nach dem gleichen Prinzip können später weitere Automatisierungen und Überwachungsfunktionen ergänzt werden.

Ein weiteres Beispiel ist die Protokollierung erfolgreicher SSH-Anmeldungen:

```
import subprocess
import socket
import re
from datetime import datetime

INFRA_SERVER = "192.168.10.20"
INFRA_PORT = 5001

# Ruft die erfolgreichen SSH-Verbindungen der letzten 24 Stunden ab
cmd = [
    "journalctl",
    "-u", "ssh",
    "--since", "24 hours ago",
    "--no-pager"
]

result = subprocess.run(
    cmd,
    capture_output=True,
    text=True
)

logs = result.stdout.splitlines()

pattern = re.compile(
    r"Accepted \S+ for (\S+) from ([0-9a-fA-F\.:]+) port (\d+)"
)

ssh_connections = []

for line in logs:
    match = pattern.search(line)
```

```
if match:
    user = match.group(1)
    source_ip = match.group(2)
    source_port = match.group(3)

    ssh_connections.append(
        f"{line}\n"
        f"Benutzer: {user}\n"
        f"Quell-IP: {source_ip}\n"
        f"Quell-Port: {source_port}\n"
        f"-----"
    )

if not ssh_connections:
    print("Keine erfolgreichen SSH-Verbindungen gefunden.")
    exit()

message = "\n".join(ssh_connections)

with socket.create_connection(
    (INFRA_SERVER, INFRA_PORT),
    timeout=10
) as sock:

    sock.sendall(message.encode("utf-8"))

print("SSH-Verbindungen an den Infrastrukturserver gesendet.")
```

In diesem Beispiel wird Port 5001 verwendet. Auf dem Infrastrukturserver muss deshalb ein entsprechender Dienst auf diesem Port lauschen, damit die Logs empfangen und gespeichert werden können.

Mögliche zukünftige Erweiterungen

In den bisherigen Beschreibungen habe ich mich bewusst auf den Netzwerkaspekt konzentriert, um die Testinfrastruktur zuerst vollständig zu verstehen und zu beherrschen.

Langfristig wäre es denkbar, Kontroll- und Monitoring-Skripte für weitere Geräte zu entwickeln. Dazu gehören beispielsweise HF-Geräte wie Empfänger. Das Spektrum wird bereits überwacht, die Geräte selbst werden jedoch – soweit technisch möglich – nicht aktiv überwacht. Dies stellt aus meiner Sicht eine relevante Lücke dar.

Auf Softwareebene könnten ausserdem benötigte Programme getestet und gewisse Aktualisierungen automatisiert werden. Dadurch liesse sich Zeit sparen und das Risiko menschlicher Fehler bei wiederkehrenden Aufgaben reduzieren. Die verfügbare Zeit könnte stattdessen stärker für gezielte Kontrollen und Verifikationen eingesetzt werden.